

MASTER MAP NAVIGATION SYSTEM



CATALOG

Getting Started (Static Map Mode)	2
Sub-Maps: Detailed Location Views	6
Getting Started (Dynamic Map Mode).....	8
MMN Settings (Edit > Project Settings > Master Map Navigation)	12
General Settings.....	12
Scene Map	14
World Map Settings.....	14
Mini Map Settings	15
Map Point.....	15
Map Generator	18
Callbacks.....	20
API.....	22
MapManeger.cs	22
MapPoint.cs	23
MapInteractive.cs	24
Controller Support.....	25

The **Master Map Navigation System** provides the following modules for your game:

- **Map Generator:** A powerful and easy-to-use tool that renders your 3D game scene into a stylized map texture. The map generator supports multi-level environments, such as tall buildings, and accommodates interiors with roofs seamlessly.
- **Mini Map:** To ensure the best performance, the Mini Map module avoids using render textures like most other packages. Instead, it converts 3D coordinates into a local 2D space to display units in the 3D world.
 - The map uses the texture generated by the **Map Generator** as its background.
 - Any object with the `<MapPoint>` component will appear as a customizable icon on the map.
 - The `<MapPoint>` component also allows you to set the icon, display name, and sub-states (e.g., quest progress).
- **World Map:** Similar to the Mini Map but with more interactive features:
 - Move and pan the map freely.
 - Toggle the visibility of map icons based on their categories.
 - Place markers to highlight specific locations or points of interest.
- **Navigation Path:** When a player right-clicks (configurable in settings) on the map, the system generates a navigation path:
 - A dynamic path line appears between the player and the target position or map icon.
 - The path is visible on both the map and in the 3D world, dynamically updating as the player or target moves.
- **Navigation Bar:** A horizontal bar at the top of the screen that helps players quickly find the direction of map icons:
 - Nearby map icons and player markers are displayed on the navigation bar.
 - When the player faces the direction of a map icon, it will appear centered on the navigation bar, providing intuitive directional guidance.

GETTING STARTED (STATIC MAP MODE)

This section is for scene-based games with pre-designed levels using pre-generated map textures. For procedurally generated worlds or seamless, massive open-world

environments using real-time render texture, please jump to the next section.

Before diving in, here are a few key things you need to know:

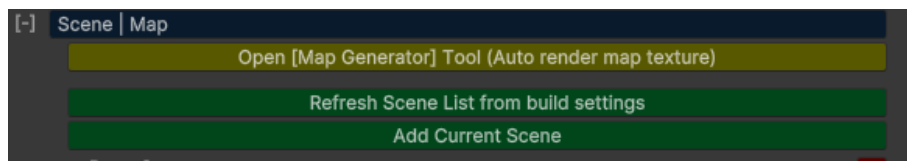
- **Demo Scene Path:** [Assets/SoftKitty/MapNavigationSystem/Demo/Scenes/DemoScene.unity](#)
- **Package Settings:** [Edit > Project Settings > Master Map Navigation](#)
- **UI Prefabs:** [Assets/SoftKitty/MapNavigationSystem/Prefabs/Interface/](#)
- **Map Creator Prefab:** [Assets/SoftKitty/MapNavigationSystem/Prefabs/MapGenerator.prefab](#)

1. Adding Your Scene to the Map System

- Open one of your game scenes.
- Go to **MNS (Master Navigation System) settings panel:** [Edit > Project Settings > Master Map Navigation](#).
- Expand the **Scene | Map** section.
- Select **Static Map** mode.



- Click **"Add Current Scene"** to add the current scene to the map system.
Alternatively, you can click **"Refresh Scene List From Build Settings"** to add all scenes from your Build Settings at once.



2. Configuring the Map Layers

- Expand the settings for the added scene.
- **(OPTIONAL)** Click **"Add New Map Layer"**. This is only needed for multi-level scenes, such as buildings with multiple floors. For flat scenes, only one layer is needed.
- Expand the **Base Layer** settings:
 - **Name:** A label for your layer (for reference only).
 - **Map Texture:** The map background texture (created in Step 3).
 - **Layer Height:** For multi-level or interior scenes, this defines the ceiling height of the layer. Ensure it matches your roof height for accurate rendering.

3. Generating the Map Texture

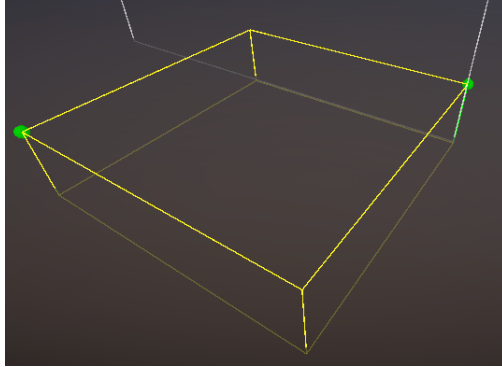
Click **"Open [Map Generator] Tool"** at the top of the settings panel to add the Map Generator prefab to your scene.

4. Adjusting the Map Area with the Map Generator

- 1) In the **Scene View**, you'll see a **giant yellow box** representing the map area.
- 2) Expand the **"Map Generator"** GameObject in the hierarchy, and you'll find two child GameObjects: **TopRight**, **BottomLeft**.

Move these two points to adjust the width and length of the map area.

- 3) In the inspector of the "Map Generator" GameObject: Adjust the **Height Range** value to ensure the box correctly covers the area intended for the map. Keep in mind: The **map must be square**. If your scene is narrow, align the box to the longest side, leaving blank areas as needed.

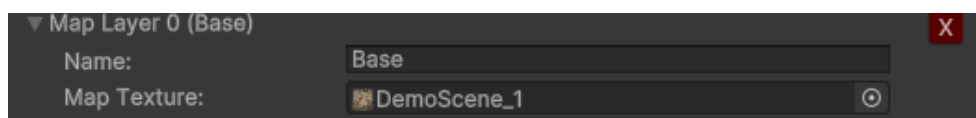


5. Configuring Map Generator Settings

- 1) In the inspector of the "**Map Generator**" GameObject, you'll find all the **render settings**. Most settings can remain as default, but set the **Layer Mask** to include all objects you want to render on the map.
- 2) Under "**Map Generator**", you'll see a GameObject called "**Ocean**": If you want blank areas to appear as **water**, adjust the Ocean GameObject's height to the water level; If not, simply **disable** the Ocean GameObject.
- 3) To reduce render time while testing settings: Set **Output Size** to **1024**, click "**Bake**" and wait approximately a minute for rendering to finish.
- 4) Once satisfied with the result: Set the final **Output Size** to **4096** (recommended), bake again—this may take longer, so grab a cup of coffee while it finishes.

6. Assigning the Map Texture

- 1) After baking, your map texture will appear in:
[Assets/SoftKitty/MapNavigationSystem/MapTextures/](#)
- 2) The texture will automatically be assigned to the **current layer settings**.



- 3) Go to the "**General Settings**" section and confirm that the "**Ground Layer Mask**" includes the correct layers used for ground and buildings in your project.



7. Adding the Map UI Prefabs

Drag the following UI prefabs from:

[Assets/SoftKitty/MapNavigationSystem/Prefabs/Interface/](#)

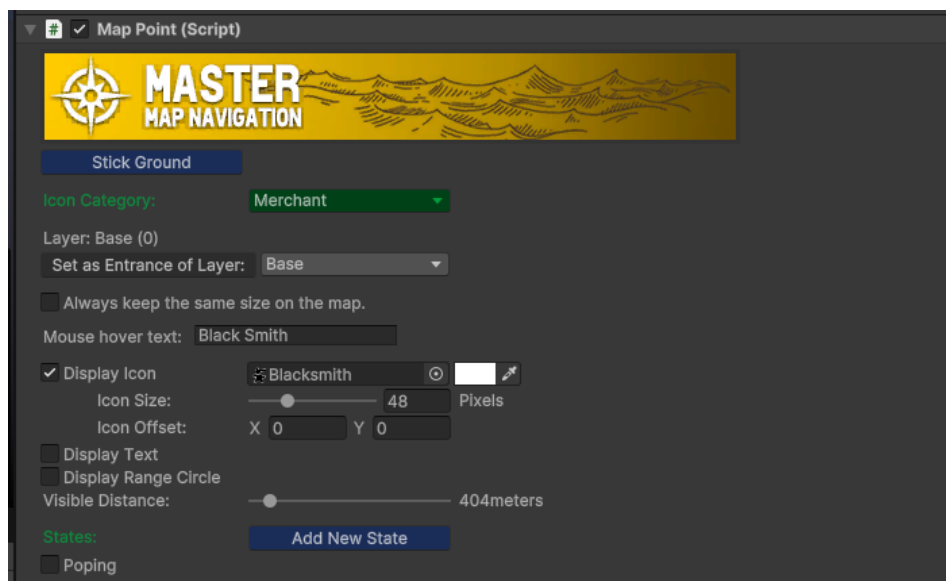
Place them on your **UI Canvas**:



The **WorldMap** prefab should remain **disabled** unless you want it to be active by default when the game starts.

8. Adding Map Points

- 1) Attach the **"MapPoint"** component to NPCs, Monsters, or Locations in your scene:
For Locations, it's best to attach the **MapPoint** component to an **empty GameObject**. This allows you to adjust its position without affecting the actual model.
- 2) Configure the **"MapPoint"** options as needed, pay attention to the **Visible Distance** setting:
This controls how close the player must be for the icon to appear on the **Navigation Bar** interface. It does not affect the map display itself.

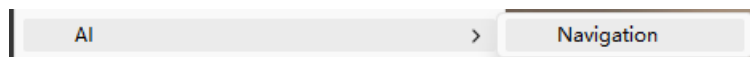


9. Baking Navigation Path (Optional)

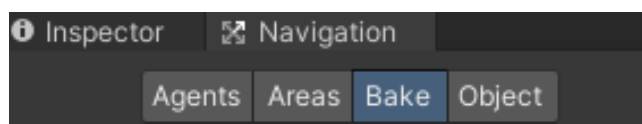
If you're using the **Navigation Path** module, bake the **Unity NavMesh** for your scene:

- **For older Unity versions:**

Access the baking panel from Window > AI > Navigation, then use the **Bake** panel.



Then use "Bake" panel to do the baking.



- **For newer Unity versions:**

Use the **NavMesh Surface** and **NavMesh Modifier** components.

Refer to Unity's documentation for details:

<https://docs.unity3d.com/Packages/com.unity.ai.navigation@2.0/manual/CreateNavMesh>.

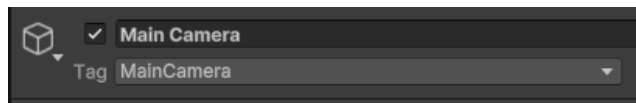
[html](#)

10. Setting the Player and Camera

- 1) The system needs to identify the player transform:
Add the following code in your player control script:

```
private void Awake()  
{  
    ...  
    MapManeger.SetPlayer(transform);  
}
```

- 2) Ensure your **main camera** has the tag "MainCamera".



- 3) Reference the **WorldMap GameObject** in your script, and bind a key or UI button to open it:

```
if (Input.GetKeyDown(KeyCode.Tab)) WorldMap.SetActive(!WorldMap.activeSelf);
```

Final Steps

Run your game and test the full functionality:

- Ensure the map loads correctly.
- Confirm **Map Points**, **Navigation Path**, and **UI interactions** work as expected.

SUB-MAPS: DETAILED LOCATION VIEWS

Beyond providing a main map texture for each scene height/layer, the Map System offers **Sub-Maps** to display highly detailed textures for specific locations within your scene. This allows for an enhanced user experience by revealing granular detail in key areas.

How Sub-Maps Work:

Sub-Maps are designed to automatically provide a more detailed view of specific points of interest in your game world.

Automatic Switching:

When the player enters a defined **Sub-Map Zone** (a rectangular bounding box you configure), the Map Interface will automatically switch from the main scene map to display the detailed **sub-map texture** of that zone.

Upon exiting the **Sub-Map** Zone, the Map Interface seamlessly switches back to the main scene map texture.

Flexible Layering:

A key advantage of **Sub-Maps** is their independent layering. While your main map might have a single layer (height), a **Sub-Map** for an indoor location like a multi-story castle can have multiple distinct layers (e.g., Basement, Ground Floor, First Floor, Rooftop), each with its own detailed texture.

Map Point Linking (for Preview):

You can link a standard **Map Point** to a specific **Sub-Map** location.

When the player clicks on such a linked **Map Point** on the main map interface, the system will temporarily switch to display the detailed **sub-map texture** of that zone, offering a preview without the player needing to physically enter the zone.

Setup Guide: Configuring Sub-Maps:

Follow these steps to define and configure **Sub-Map** zones for your scenes:

- Navigate to the "**Scene | Map**" section within the Map package settings in your Unity Editor.
- Locate and expand the "**Detailed Location Sub-Maps**" settings.
- Click the "**Add New Sub-Map**" button. A new sub-map entry will appear for the currently selected scene.
- Configure the following properties for your new **Sub-Map**:
 - (Sub-Map) UID**: Enter a unique identifier for this **Sub-Map** within the current scene. This ID is used for internal referencing and linking.
 - (Sub-Map) Display Title**: Provide a descriptive title. This title will be displayed on the Map Interface when the player is inside this **Sub-Map** Zone.
 - (Sub-Map) Add New Map Layer**: Click this button to add map layers specific to this **Sub-Map**. Set up each layer's properties (e.g., texture resolution, scale, and offset) in the same manner as you would for main map layers.

Baking Sub-Map Textures:

Once your **Sub-Map** zones and layers are configured, you'll need to bake their detailed textures:

- Open the **Map Creator** tool.
- Locate the "**Bake for Sub-Map**" checkbox and enable it.
- Use the left and right arrow buttons adjacent to this checkbox to browse through the existing **Sub-Map** locations you've defined.
- Select the specific **Sub-Map** you wish to bake the texture for.
- Proceed to bake the texture for the selected **Sub-Map**'s layers, following the same process as introduced for baking main map textures.

- After baking, the generated texture will be automatically assigned to the corresponding layers of the selected **Sub-Map** Zone.

Linking Map Points to Sub-Maps for Preview

To enable interactive previewing of **Sub-Maps** directly from the main map:

- Select the **MapPoint** component on the GameObject you wish to link.
- Locate and check the "**Click to open sub-map**" checkbox within the **MapPoint** settings.
- Use the left and right arrow buttons (or dropdown selector, if available) to choose the target **Sub-Map** Zone that this **MapPoint** should preview when clicked.

GETTING STARTED (DYNAMIC MAP MODE)

This section is for procedurally generated worlds or seamless, massive open-world environments using real-time render texture. For scene-based games with pre-designed levels using pre-generated map textures, please scroll up to the previous section.

Before diving in, here are a few key things you need to know:

- **Demo Scene Path:** [Assets/SoftKitty/MapNavigationSystem/Demo/Scenes/DemoScene.unity](#)
- **Package Settings:** [Edit > Project Settings > Master Map Navigation](#)
- **UI Prefabs:** [Assets/SoftKitty/MapNavigationSystem/Prefabs/Interface/](#)
- **Map Creator Prefab:** [Assets/SoftKitty/MapNavigationSystem/Prefabs/MapGenerator.prefab](#)

1. **Switch to Dynamic Map mode**
2. **Open one of your game scenes.**

Go to **MNS (Master Navigation System) settings panel:** [Edit > Project Settings > Master Map Navigation](#).

Expand the **Scene | Map** section.

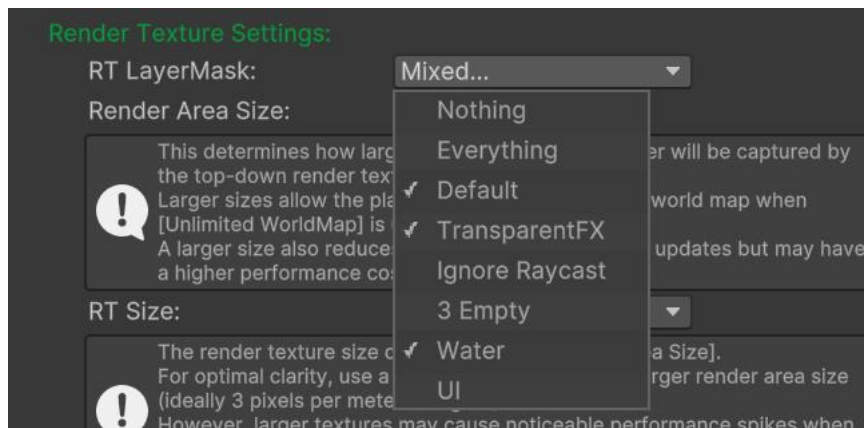
Select **Dynamic Map** mode.



3. **Select Render Texture LayerMask**

Choose the layers to be rendered into the map's render texture. Ensure that layers containing

characters or dynamically moving objects are excluded.



4. Set Render Area Size

This determines how large the area around the player will be captured by the top-down render texture camera. Larger sizes allow the player to scroll further on the world map when [Unlimited WorldMap] is unchecked. A larger size also reduces the frequency of minimap updates but may have a higher performance cost.



5. Set Render Texture Size

The render texture size depends on the [Render Area Size]. For optimal clarity, use a larger texture size with a larger render area size (ideally 3 pixels per meter or higher). However, larger textures may cause noticeable performance spikes when the minimap updates. (The minimap only updates when the player moves out of the previously rendered area.)



6. Unlimited WorldMap

If your game is massive open world, and you want the player be able to scroll to anywhere instead of limited with the areas around., you can enable this feature. When enabled, players can scroll the map to any position in the 3D world. Ensure that in your script, the 3D scene within [*MapInteractive.WorldMapInstance.ViewRect*](#) is always loaded to match the visible map area.

7. Setup the RT Camera Height and Depth

The render texture camera height must be above the tallest objects in your scene to avoid clipping. Its depth should cover the lowest point in your scene from the camera's height.

For example

If the highest object in your game is at 200 on the Y-axis, and the lowest point is at -50, set the RT Camera Height to 201 and the RT Camera Depth to 251.

Avoid using unnecessarily large values, as they may cause issues with the stylized rendering and reduce performance efficiency.

8. Choose Stylized Rendering

To enable stylized rendering for the render texture:

- Check the "**Stylized Rendering**" checkbox.
- In the "**RT Material**" field, select a material from:

[Assets/SoftKitty/MapNavigationSystem/Materials/CustomRenderTexture/](#).

When using these materials, ensure the "**Depth Only**" option (located below the "**RT Material**" setting) is also checked.

If you'd like to create your own custom render texture shader:

- Use Shader Graph to create a custom shader.
- Create a new material and assign your custom shader to it.
- Assign the

[Assets/SoftKitty/MapNavigationSystem/Textures/System/DynamicMapSourceRT](#)

to the "**InputTex**" slot of the material.

- Finally, assign your new material to the "**RT Material**" field.

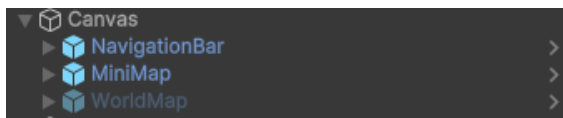
This allows for full customization of the map's visual style.

9. Adding the Map UI Prefabs

Drag the following UI prefabs from:

[Assets/SoftKitty/MapNavigationSystem/Prefabs/Interface/](#)

Place them on your **UI Canvas**:

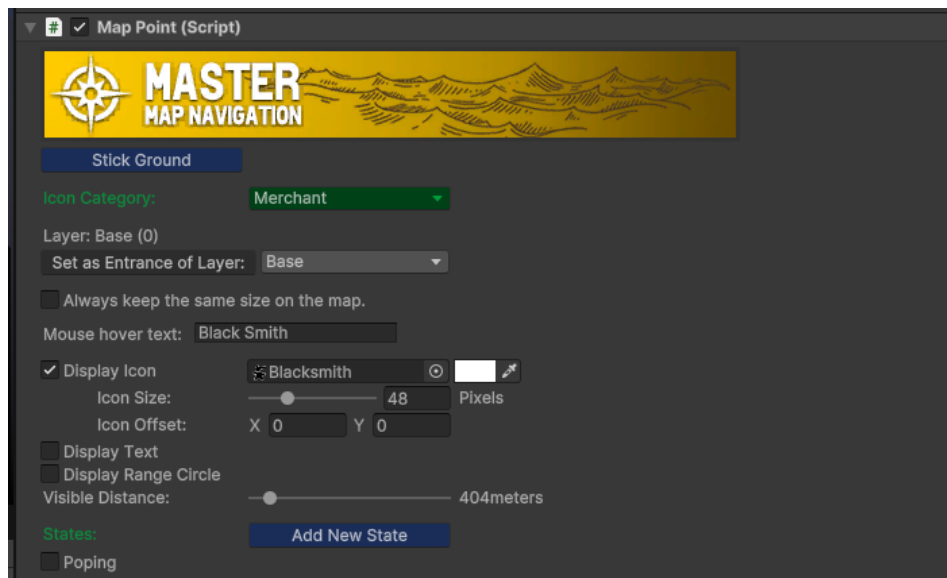


The **WorldMap** prefab should remain **disabled** unless you want it to be active by default when the game starts.

10. Adding Map Points

- Attach the "**MapPoint**" component to NPCs, Monsters, or Locations in your scene:
For Locations, it's best to attach the **MapPoint** component to an **empty GameObject**.
This allows you to adjust its position without affecting the actual model.
- Configure the "**MapPoint**" options as needed, pay attention to the **Visible Distance** setting:
This controls how close the player must be for the icon to appear on the **Navigation Bar**

interface. It does not affect the map display itself.

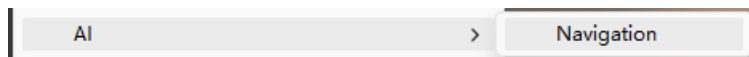


11. Baking Navigation Path (Optional)

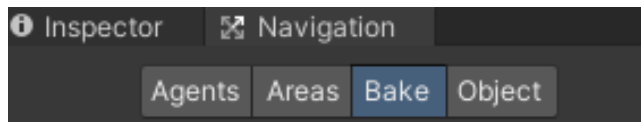
If you're using the **Navigation Path** module, bake the **Unity NavMesh** for your scene:

- **For older Unity versions:**

Access the baking panel from Window > AI > Navigation, then use the **Bake** panel.



Then use "Bake" panel to do the baking.



- **For newer Unity versions:**

Use the **NavMesh Surface** and **NavMesh Modifier** components.

Refer to Unity's documentation for details:

<https://docs.unity3d.com/Packages/com.unity.ai.navigation@2.0/manual/CreateNavMesh.html>

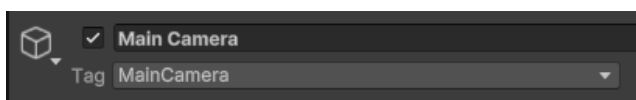
12. Setting the Player and Camera

The system needs to identify the player transform:

Add the following code in your player control script:

```
private void Awake()
{
    MapManeger.SetPlayer(transform);
}
```

Ensure your **main camera** has the tag "**MainCamera**".



Reference the **WorldMap GameObject** in your script, and bind a key or UI button to open it:

```
if (Input.GetKeyDown(KeyCode.Tab)) WorldMap.SetActive(!WorldMap.activeSelf);
```

Final Steps

Run your game and test the full functionality:

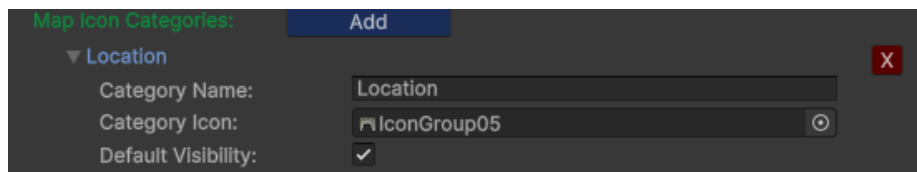
- Ensure the map loads correctly.
- Confirm **Map Points**, **Navigation Path**, and **UI interactions** work as expected.

MMN SETTINGS (EDIT > PROJECT SETTINGS > MASTER MAP NAVIGATION)

GENERAL SETTINGS

➤ Map Icon Categories:

Add or remove categories for map icons here.



In the **MapPoint** inspector, you can assign each icon to a specific category.



➤ Player's Icon:

Set the texture for the player's map icon. Ensure the texture is **transparent** and has **MipMaps disabled**.

➤ Ground Layer Mask:

Define the **layer mask** for ground and buildings. This mask will be used for raycasting the ground height when creating markers or navigation paths on the map.

Custom Markers

Custom Marker system is for players to be able to place different map icons on the map to mark the location they're interested. Enable this feature will allow you to setup a list of custom markers for player to select from.

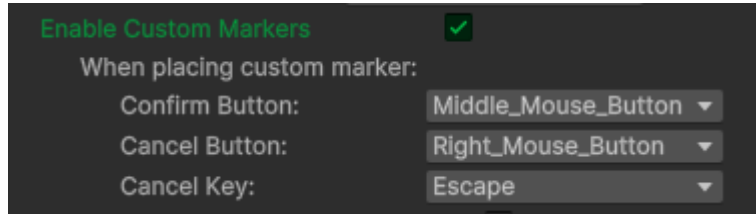
On the bottom right of the **WorldMap** UI, there is a yellow button to toggle the custom marker list:



After player select one of the icon in the list, a hint message will show up to indicate which buttons to press for **confirm** or **cancel**:



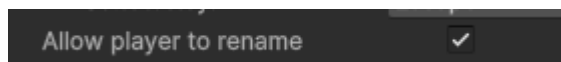
The buttons can be modified below the **Enable Custom Markers** settings:



Players will also be able to **rename** the markers they placed by clicking the marker:



This feature can be enabled by toggle **'Allow player to rename'**:



To save/load custom markers players placed, please use the following API:

Save: `string _json = MapManeger.ExportCustomMarkerJsonData();`

Load: `MapManeger.ImportCustomMarkerJsonData(string _json)`

Example Code:

```
void LoadCustomMarker()
{
    if (File.Exists("E:/MarkerText.txt")){
        string _json = File.ReadAllText("E:/MarkerText.txt", System.Text.Encoding.UTF8);
        MapManeger.ImportCustomMarkerJsonData(_json);
    }
}

void SaveCustomMarker(){

    string _json = MapManeger.ExportCustomMarkerJsonData();
    File.WriteAllText("E:/MarkerText.txt", _json, System.Text.Encoding.UTF8);

}
```

SCENE | MAP

- **Map Mode**

use the **Static Map** Mode to render stylized 2D maps for scene-based games with pre-designed levels. For procedurally generated worlds or seamless, massive open-world environments, switch to the **Dynamic Map** Mode with real-time render textures and customizable stylized shaders.
- **Open [Map Generator] Tool (Render Map Texture)**

Create or locate the **Map Generator** prefab in the current scene.
- **Refresh Scene List From Build Settings**

Automatically adds all scenes from your **Build Settings** to the list. Remove any scenes that don't require the map system, such as your title screen.
- **Add Current Scene**

Add the currently opened scene to the list.
- **Display Title:**

Set the name of the scene to display on the map.
- **Map Layers:**

For **multi-level scenes** (e.g., buildings with multiple floors), add and configure layers. For flat scenes, only one layer is required.

 - **Name:** Internal name for the layer (not displayed in the game).
 - **Map Texture:** Assign the map background texture created using the **Map Generator** tool.
 - **Layer Height:** Define the ceiling height for this layer. For interior scenes, set this value under the roof to ensure proper map rendering.
- **Bottom Left Corner Position/Top Right Corner Position:**

Define the rectangular area of the map. These values are automatically set when using the **Map Generator** tool.

WORLD MAP SETTINGS

- **Default Zoom:**

The default zoom level when the world map is opened.
- **Lock Icon Scale:**

Maintain consistent icon sizes while zooming in/out. To set this per icon, uncheck this option and configure it in the **MapPoint** inspector using.

A dark grey rectangular button with a white checkmark icon on the left and the text "Always keep the same size on the map." in a light grey font.
- **Display Scene Title:**

Show or hide the title of the current scene on the world map interface.
- **Enable Navigation Path:**

Enable or disable the navigation path system.

➤ **Start Navigation Mouse Button:**

Define the mouse button used to start navigation by clicking on the map.

➤ **Map Scroll Left/Right/Up/Down Key:**

Assign keyboard keys for scrolling the map.

➤ **Create Map Marker Key+Mouse Button:**

Allow players to create map markers:

- Hold a specific **keyboard key** and click with the **mouse button** to add a marker.
- Clicking the same mouse button on an existing marker (without holding the key) will remove the marker.

MINI MAP SETTINGS

➤ **Default Zoom:**

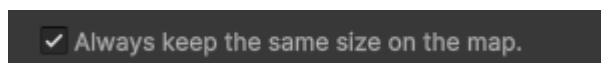
Set the default zoom level for the mini map.

➤ **Maximum Zoom Out:**

Define the maximum zoom-out level for the mini map.

➤ **Lock Icon Scale:**

Maintain consistent icon sizes while zooming in/out. To set this per icon, uncheck this option and configure it in the **MapPoint** inspector using “Always keep the same size.”



➤ **Display Scene Title:**

Show or hide the title of the current scene on the mini map interface.

➤ **Display Coordinate:**

Enable or disable the display of the player's coordinates on the mini map interface.

➤ **Enable Player Orientation Button:**

Enable the **compass button** on the top-left of the mini map. This allows players to toggle whether the map rotates to align with the player's direction.

➤ **Player Orientation as Default:**

Set the mini map to always rotate by default, keeping the player's direction at the top.

➤ **Allow Zoom In/Out:**

Enable or disable zooming in/out on the mini map using the mouse scroll wheel.

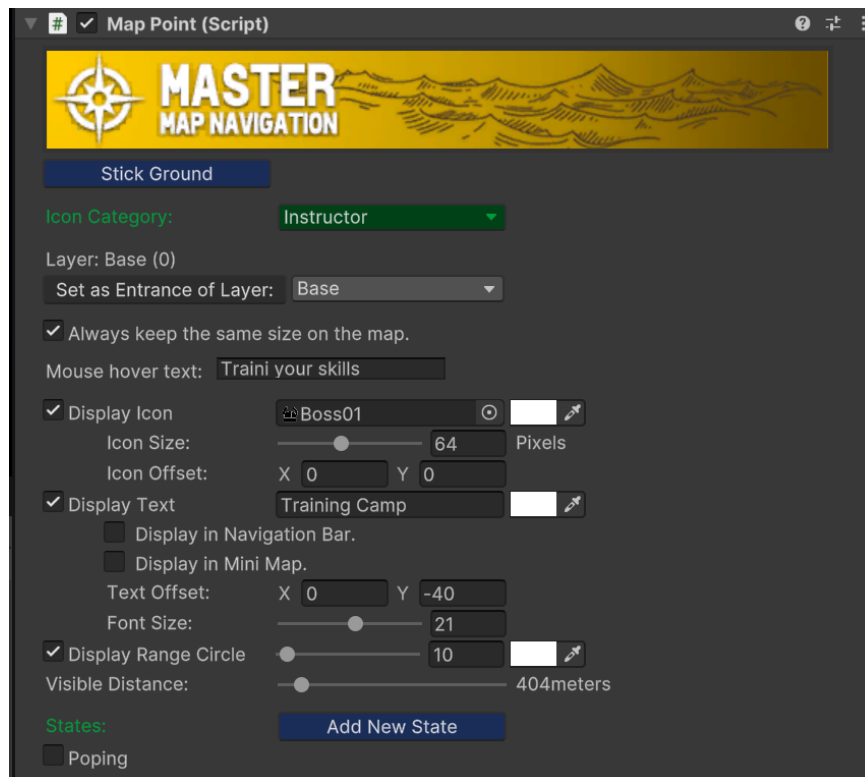
MAP POINT

MapPoint.cs is a versatile component that integrates a **GameObject** into the map system and manages its behavior. Any **GameObject** with a **MapPoint** component will appear on the **Mini Map**, **World Map**, and **Navigation Bar** based on its position in the game scene.

Map points can be placed manually in the scene or dynamically instantiated at runtime using prefabs.

To enable or disable a map point's visibility on the map and navigation bar, use:

```
<MapPoint>().SetVisible(bool _visible);
```



➤ **Stick Ground:**

Click this button to snap the GameObject to the ground level.

Useful when placing land markers to align with the terrain without manually adjusting the height.

➤ **Icon Category:**

Assign the map point to a category set up in the **MMN Settings**.

Players can toggle the visibility of different categories in the **World Map** interface.

➤ **Layer:**

Determines which map layer this point belongs to based on its Y-axis position.

Example: First floor: height 0–3; Second floor: height 3–6. If a map point has a Y position of 3.5, it will belong to the second layer.

➤ **Set as Entrance of Layer:**

Define entrance points for multi-level scenes to enable navigation across layers.

In buildings with multiple floors, the navigation path will lead players to the layer entrance first before navigating to the destination.

Setup:

- 1) Create a map point.

- 2) Use the **“Set as Entrance of Layer”** button and select the correct target layer from the dropdown.



➤ **Always keep the same size on the map:**

Prevents the map icon from scaling with zoom levels.

This setting overrides the general **Lock Icon Scale** option in **MMN Settings**.

➤ **Mouse Hover Text:**

Displays a hint text when the player hovers the cursor over the map icon.

Leave the text field blank to disable this feature.

➤ **Display Icon:**

Controls the visibility and appearance of the map icon.

Customizable Options: Texture, Color, Size, and Offset.

Related Api:

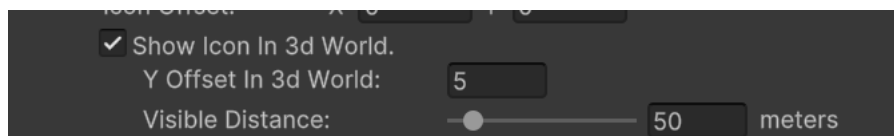
<MapPoint>().ToggleIcon(bool _enable); //Toggle the icon visibility via script

<MapPoint>().SetIcon(Texture2D _icon); // Set the icon texture via script

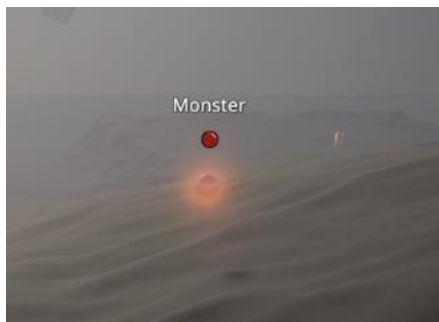
<MapPoint>().SetIconColor(Color _color); //Set the icon color via script

<MapPoint>().SetIconSize(int _pixelSize); //Set the icon pixel size via script

➤ **Show Icon in 3d World:**



if this checkbox is checked, an icon of this map point will be displayed on top of the 3d object like this:



➤ **Display Text:**

Controls the visibility and appearance of the map text.

Customizable Options: Content, Color, Font Size, and Offset. Option to display the text on the **Mini Map** or **Navigation Bar**.

Related Api:

<MapPoint>().ToggleText (bool _enable); //Toggle the text visibility via script

<MapPoint>().SetText(string _text); // Set the text content via script

<MapPoint>().SetTextColor (Color _color); //Set the text color via script

<MapPoint>().SetTextSize (int _fontSize); //Set the text font size via script

➤ **Display Range Circle:**

Displays a circular range around the map point with customizable radius and color.

Useful for marking areas, such as quest regions or item drop zones.

Related Api:

<MapPoint>(). ToggleRange (bool _enable); //Toggle the range visibility via script

<MapPoint>(). SetRangeRadius (float _radius); //set the radius of the range via script

➤ **States:**

Displays state icons (e.g., quest status or attention marks) on the top-right corner of the map icon.

Related Api:

<MapPoint>().State = x ; //When set state as -1 will hide the state icon.

➤ **Popping:**

Makes the icon “pop” on the map to attract the player’s attention.

Related Api:

<MapPoint>().SetPoping(bool _enable);

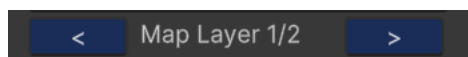
MAP GENERATOR

The **Map Generator** is a powerful tool that converts your 3D scene into a stylized 2D map texture.

Below are the key points you need to know to use it effectively:

- **Layer Mask:** Only **GameObjects** and **Terrain** within the **Layer Mask** set in the **Map Generator** will be rendered.
- **Colliders:** Only **GameObjects** and **Terrain** with colliders will be rendered.
- **Render Area:** Only objects inside the **yellow box** defined by the **Map Generator** will be rendered. The yellow box (render area) is controlled by: **Bottom Left** and **Top Right** transforms. **Height Range** settings.

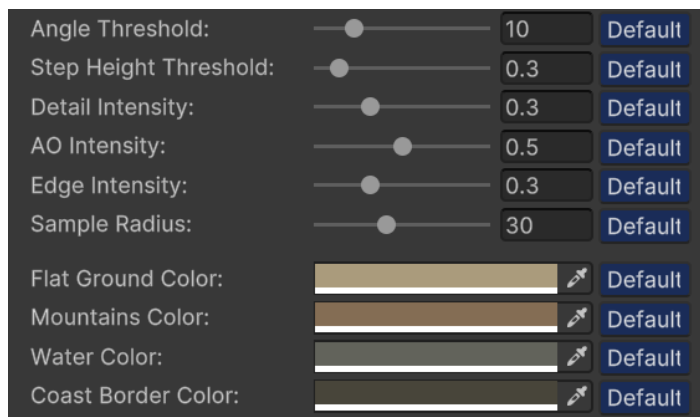
For **multi-layer scenes** (e.g., multi-story buildings), you should have multiple **Map Generator** prefabs in the scene, each corresponding to a specific layer.



Use the **Layer Setting** to define the layer of each **Map Generator** instance and adjust its height range accordingly.

The **Map Generator** offers various stylization options to customize the appearance of your map.

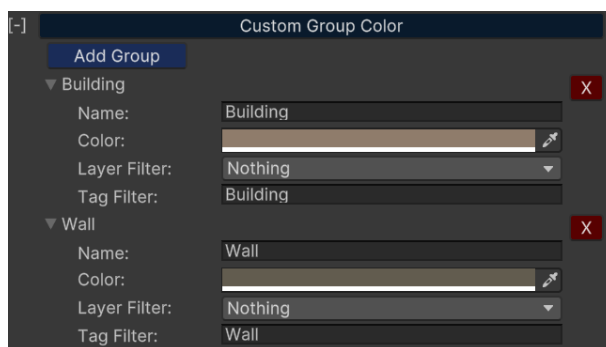
These options are self-explanatory by their name:



Custom Group Color:

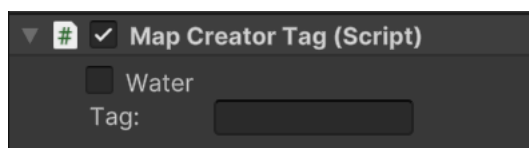
➤ Setup Custom Groups:

You can create multiple **Custom Groups** to filter GameObjects and apply unique colors to them during rendering.



➤ Filtering with Tags:

If you prefer not to change the **layer** or **tag** of your GameObjects, you can use the **MapCreatorTag** component.



Add the **MapCreatorTag** component to the GameObject and set the desired **Tag** string. The Map Generator will recognize this tag and apply the appropriate color based on the **Custom Group** settings.

Tip: If you want a GameObject to be treated as **water**, check the “**Water**” checkbox in the **MapCreatorTag** component.

➤ Ocean

The **Ocean** GameObject represents the ocean surface for your map. Adjust its size and water level by **scaling** and **moving** the transform.

➤ Lake

The **Lake** GameObject can be adjusted to create water bodies like lakes or ponds.

- Customize the lake's shape by **moving the green nodes** around it.
- Duplicate the **Lake** GameObject to add multiple lake areas on your map.



CALLBACKS

All callbacks in the system are managed by the **MapManager.cs** script. Below is the list of available callbacks and their usage, including script examples for each.

MapPointStateChangeCallback

Signature: *MapPointStateChange(Transform _key, int _state)*

Trigger: When the state variable of a **MapPoint** changes.

Usage Example:

```
MapManeger.MapPointStateChangeCallback += OnMapStateChange;  
public void OnMapStateChange(Transform _key, int _state){}
```

MapPointCreateCallback

Signature: *MapPointCreate(Transform _key, MapPointData _data, int _state)*

Trigger: When a **MapPoint** is added to the map system.

Usage Example:

```
MapManeger.MapPointCreateCallback += OnMapPointCreate;  
public void OnMapPointCreate(Transform _key, MapPointData _data, int _state){}
```

MapPointRemoveCallback

Signature: *MapPointRemove(Transform _key)*

Trigger: When a **MapPoint** is removed from the map system.

Usage Example:

```
MapManeger.MapPointRemoveCallback+= OnMapPointRemove;  
public void OnMapPointRemove (Transform _key){}
```

MapPointDataUpdateCallback

Signature: *MapPointDataUpdate(Transform _key, MapPointData _data)*

Trigger: When the data of a **MapPoint** is updated.

Usage Example:

```
MapManeger. MapPointDataUpdateCallback += OnMapPointDataUpdateCallback;  
public void OnMapPointDataUpdateCallback (Transform _key, MapPointData _data){}
```

NavigationPathCallback

Signature: *NavigationPathUpdate(List<Vector3> _path,bool _updateAll)*

Trigger: When a navigation path is created, removed, or updated.

Usage Example:

```
MapManeger. NavigationPathCallback += OnNavigationPathUpdate;  
public void OnNavigationPathUpdate (List<Vector3> _path,bool _updateAll){}
```

When _updateAll is false, means only the first and last position of the List has changed, you should pull the first and last value of the list to replace the current one. When the List is empty means the path has been removed.

MapIconClickCallback

Signature: *MapIconClick(MapPoint _point, int _button)*

Trigger: When a player clicks on a map icon.

Usage Example:

```
MapManeger. MapIconClickCallback += OnMapIconClick;  
public void OnMapIconClick (MapPoint _point, int _button){}
```

MapLayerChangeCallback

Signature: *MapLayerChange(int _layer)*

Trigger: When the player enters a different map layer.

Usage Example:

```
MapManeger. MapLayerChangeCallback+= OnMapLayerChange;  
public void OnMapLayerChange (int _layer){}
```

MarkerOutLimitCallback

Signature: *MarkerOutLimit(Vector3 _worldPos)*

Trigger: When the player try to place more marker than the maximum allowed number.

Usage Example:

```
MapManeger.MarkerOutLimitCallback+= OnMarkerOutLimit;  
public void OnMarkerOutLimit(Vector3 _worldPos) { }
```

API

MapManeger.cs

- `public static void SetPlayer(Transform _playerTransform)`
Assigns the player's transform to the Master Map Navigation system.
This function **must be called** when the scene starts to initialize the player's position.
- `public static void NavigateToHere(Vector3 _worldPos)`
Creates a navigation path from the player's current position to the specified world space position.
_worldPos: The target position in world space.
- `public static void StopNavigation()`
Clears the current navigation path and stops the navigation.

- `public static MapPoint PlaceMarker(Vector3 _worldPos)`
 Adding a custom marker on the map to mark points of interest.
 Places a marker on the map at the specified world space position.
 Return the MapPoint component representing the created marker.
- `public static void RemoveMarker(MapPoint _point)`
 Removes a previously placed marker from the map.
`_point`: The MapPoint component of the marker to be removed.
- `public static int GetLayerByPosition(Vector3 _pos)`
 Retrieves the layer ID of the map based on a given world space position. The layer ID where the position resides.
`_pos`: The target position in world space.
- `public static List<MapPoint> GetLayerEntrance(int _ownLayer, int _targetLayer)`
 Retrieves a list of entrance points to move from one map layer `_ownLayer` to another `_targetLayer`.
`_ownLayer`: The current layer ID. `_targetLayer`: The destination layer ID.
- `public static MapPoint GetLayerClosestEntrance(int _ownLayer, int _targetLayer)`
 Finds the closest entrance point between one map layer `_ownLayer` to another `_targetLayer`.
`_ownLayer`: The current layer ID. `_targetLayer`: The destination layer ID.

MapPoint.cs

- `public int State`
 Get/Set the state of this map point. Setting the value to -1 will hide the state icon.
- `public void SetPopping(bool _enable)`
 Enables or disables the "popping" effect of the map icon to draw the player's attention.
- `public void SetVisible(bool _visible)`
 Toggles the visibility of the map point.
- `public void ToggleText(bool _enable)`
 Enables or disables the visibility of the text associated with the map point.
- `public void SetText(string _text)`
 Sets the content of the text displayed for the map point.
- `public void SetTextColor(Color _color)`
 Sets the color of the text.
- `public void SetTextSize(int _fontSize)`
 Sets the font size of the text.
- `public void ToggleRange(bool _enable)`
 Enables or disables the visibility of the range circle around the map point.

- `public void SetRangeRadius(float _radius)`
Sets the radius of the range circle.
- `public void SetRangeColor(Color _color)`
Sets the color of the range circle.
- `public void ToggleIcon(bool _enable)`
Enables or disables the visibility of the map icon.
- `public void SetIcon(Texture2D _icon)`
Sets the texture of the map icon.
- `public void SetIconColor(Color _color)`
Sets the color of the map icon.
- `public void SetIconSize(int _pixelSize)`
Sets the pixel size of the map icon.

MapInteractive.cs

- `public static MapInteractive MiniMapInstance;`
The instance of the MiniMap ui.
- `public static MapInteractive WorldMapInstance;`
The instance of the WorldMap ui.
- `public Rect ViewRect;`
With **Dynamic Map** mode, when “**Unlimited WorldMap**” is enabled, players can scroll the map to any position in the 3D world. Ensure that in your script, the 3D scene within `MapInteractive.WorldMapInstance.ViewRect` is always loaded to match the visible map area.
- `public Vector3 Focus(Vector3 _worldPosition, Vector2 _center)`
This method moves the map to focus on a specific 3D world position.
 - `_worldPosition`: The 3D world position to focus on.
 - `_center`: A normalized screen position (ranging from 0 to 1) that determines where the focused position will appear on the map.

For example, to center the player’s position on the map, you would use:
`Focus(Player.position, new Vector2(0.5f, 0.5f));`
 Here, (0.5f, 0.5f) represents the center of the screen. Use other values to place the position elsewhere (e.g., (0, 0) for the bottom-left corner or (1, 1) for the top-right corner).
- `public void Zoom(float _value, Vector2 _focus)`
This method zooms the map in or out by a specified value and optionally adjusts the focus point.
 - `_value`: Determines the zoom level. Positive values zoom in. Negative values zoom out.
 - `_focus`: A normalized screen position (ranging from 0 to 1) that specifies where the zoom will

focus.

For example, (0.5f, 0.5f) represents the center of the screen.

Use other values (e.g., (0, 0) for bottom-left or (1, 1) for top-right) to focus on a different part of the map.

If you don't need to focus on a specific position, pass new Vector2(0.5f, 0.5f) to keep the zoom centered on the screen.

- `public void SwitchMapLayer(int _layer)`
Switches the map to a specified layer. This only affects the current map instance interface; it does not alter the player's position or any other map instance.
- `public void Compass()`
Toggles **Player Orientation Mode**, aligning the player's direction to always point toward the top of the interface.
- `public static Vector3 WorldToMapPosition(Vector3 _worldPos)`
Converts a world space position to a map space position. The resulting map position is 2D, so the Y-axis value will always be 0.
- `public static Vector3 MapToWorldPosition(Vector3 _mapPos)`
Converts a map space position back to a world space position. The Y-axis value remains 0, as the map space is strictly 2D.
- `public Vector3 TransferPos(Vector3 _pos, RectTransform _parentTransform)`
Transfrom mouse position to map position.

CONTROLLER SUPPORT

If your game is designed for controller, Inherit your script from [ControllerMapping.cs](#), replace the one attached on the map interface prefab, and assign it to the '**ControllerMappingScript**' slot of the [<MapInteractive>](#) component.

Then override the below functions with your controller code:

- `public virtual void OnMapInit(MapInteractive.MapTypes _type)`
When map gets enable, this function will be called.
- `public virtual void SetVirtualCursorVisible(bool _visible)`
Set the visibility of the Virtual Cursor.
- `public virtual void SetVirtualCursorPosition(Vector3 _pos)`
Override the position of the Virtual Cursor.
- `public virtual void ControlVirtualCursor()`
Code for control the Virtual Cursor in your way.
- `public virtual bool isVirtualCursorOver(RectTransform _rectTrans)`
Retrieve whether a rect transform is overlapping with the Virtual Cursor

- `public virtual Vector3 GetPointerPosition()`
Retrieve the position of the mouse pointer, if Virtual Cursor is enabled, return the position of the Virtual Cursor insteaded.
- `public virtual bool GetZoomInKey()`
Retrieve whether the zoom in button is pressed, override your controller button press code here.
- `public virtual bool GetZoomOutKey()`
Retrieve whether the zoom out button is pressed, override your controller button press code here.
- `public virtual bool GetPlaceMarkerKey()`
Retrieve whether the place map marker button is pressed, override your controller button press code here.
- `public virtual string GetPlaceMarkerKeyHint()`
Retrieve the hint text for place map marker button, override your controller button name here.
- `public virtual bool GetRemoveMarkerKey()`
Retrieve whether the remove map marker button is pressed, override your controller button press code here.
- `public virtual string GetRemoveMarkerKeyHint()`
Retrieve the hint text for remove map marker button, override your controller button name here.
- `public virtual bool GetNavigateKey()`
Retrieve whether the navigate button is pressed, override your controller button press code here.
- `public virtual string GetNavigateKeyHint()`
Retrieve the hint text for navigate button, override your controller button name here.
- `public virtual bool GetScrollUpKey()`
Retrieve whether the scroll map up button is pressed, override your controller button press code here.
- `public virtual bool GetScrollDownKey()`
Retrieve whether the scroll map down button is pressed, override your controller button press code here.
- `public virtual bool GetScrollLeftKey()`
Retrieve whether the scroll map left button is pressed, override your controller button press code here.
- `public virtual bool GetScrollRightKey()`
Retrieve whether the scroll map right button is pressed, override your controller button press code here.